

Efficient Seeding of Random Number Engines

Document number: P0205R1

Date: 2021-06-04

Project: Programming Language C++

Audience: Study Group 6 (Numerics), Library Evolution Working Group, Library Working Group

Reply-to: Moritz Klammler <moritz@klammler.eu> (OpenPGP: 2732 DA32 C8D0 EEEC A081 BE9D CF6C 5166 F393 A9C0)

Table of Contents

- 1 Introduction
- 2 Motivation and Scope
 - 2.1 Background
 - 2.2 The Problem with the Current Standard Library
 - 2.3 The Proposed Solution
- 3 Impact on the Standard
 - 3.1 Refinements and Additions to the Standard Library
 - 3.2 Clarification on Random Number Engine Requirements
 - 3.3 Deprecation of Existing Features
- 4 Design Decisions
- 5 Alternative Solutions
- 6 Implementations
- 7 Proposed Wording
- Acknowledgments
- Changelog
- References

1 Introduction

The purpose of this proposal is to make properly seeding a random number engine in a (non-)deterministic way easy and fast.

In order to achieve this, the following changes are proposed:

- Introduction of a new named requirement *seed generator*, which is a relaxation of *seed sequence* [\[rand.req.seedseq\]](#).
- Addition of a new helper type `std::seed_adapter` which makes any *uniform random bit generator* [\[rand.req.urng\]](#) usable as a *seed generator*.

The changes are non-breaking and can be implemented as a pure library solution with current C++20 features.

2 Motivation and Scope

2.1 Background

C++11 introduced a powerful random number generation library [\[rand\]](#) under the `<random>` header. It provides *random number engines* [\[rand.req.eng\]](#) that can be used either directly as a source of uniformly distributed pseudo random integers of unsigned type or together with *random number distributions* [\[rand.req.dist\]](#) to produce pseudo random numbers according to a variety of distributions.

If an engine has an internal state of N bits, it can produce at most 2^N different sequences and the longest sequence it can produce will have a period of at most 2^N values until it necessarily starts repeating itself. Applications that wish to produce high-quality pseudo random numbers will therefore choose an engine such as the Mersenne twister (e.g. `std::mt19937`) with a large internal state. Choosing an engine with a large internal state helps ensuring that the period of the generated sequence will be large. However, in order to ensure that the number of different sequences the engine can produce is also exponential in the size of its state, it has to be made sure that the initial state is evenly distributed across

all possible states. This is done by *seeding* the random number engine. If each of the 2^N states should be chosen with equal probability as the initial state, seeding requires 2^N bits of entropy.

The standard library provides the type `std::random_device` [\[rand.device\]](#), a *uniform random bit generator* [\[rand.req.urng\]](#) that is supposed (but, unfortunately, not required) to produce a non-deterministic sequence of uniformly distributed integers of type `unsigned int`. The natural choice for an application that wishes to generate pseudo random numbers and does not need (and often doesn't want) reproducibility is to use it for seeding a random engine that is then used to produce pseudo random numbers. In addition, the standard library provides the type `std::seed_seq` [\[rand.util.seedseq\]](#) which implements the *seed sequence* [\[rand.req.seedseq\]](#) requirements (and is the only type in the current standard library doing so). It can be used to seed a random number engine using entropy obtained from an arbitrary sequence of integers (e.g. a user-provided byte string read from a file or an environment variable to allow reproducing experiments with the same seed).

2.2 The Problem with the Current Standard Library

Unfortunately, the current standard library does not provide any convenient way to use a `std::random_device` to properly (in the sense that each initial state is equally likely) seed a random engine. Nor does it make doing so with a custom uniform random bit generator or writing a custom seed sequence easy.

The naïve approach that most people seem to use is drawing one random number from `std::random_device` (or whatever source of entropy they have) and then use the constructor which takes a single integer as seed.

```
template <typename EngineT>
void seed_non_deterministically_1st(EngineT& engine)
{
    std::random_device device{};
    engine.seed(device());
}
```

This code is severely flawed. If `EngineT` is `std::mt19937`, it has a state size of 19968 bits. However, if an `unsigned int` is 32 bits (as is the common case on many platforms today), then of the up to 2^{19968} states, at most 2^{32} (that is one 2^{-19936} -th) can possibly be chosen!

To illustrate that this is in fact a real problem, O'Neill [\[1\]](#) has pointed out that a `std::mt19937` engine seeded like above can never produce certain integers as its first value. This can have had consequences on real-world programs. A program that incorrectly assumes that an impossible number will eventually be produced as the engine's first (or n -th) output is broken in a very subtle way. After all, it would take extensive and practically unrealistic unit testing to do an exhaustive search over the possible seeds and even detect the bug.

In addition to seeding an engine with an integer, the standard library also provides a way to seed it with a so-called *seed sequence* [\[rand.req.seedseq\]](#). A seed sequence may be constructed in a deterministic way from zero or more integers that provide some initial entropy. The numbers are then possibly scrambled and stored in some internal state of the seed sequence which can be externalized using the `param` member function. Such a memento may then be used to re-create a seed sequence of the same type in the same state. A seed sequence provides a member function `generate` that takes a pair of random access iterators and assigns a uniformly distributed unsigned 32 bit integer to each element in the range denoted by the iterator pair. The standard library provides a single implementation of a seed sequence in `std::seed_seq` [\[rand.util.seedseq\]](#). This type can be used to seed a random engine more thoroughly.

```
template <typename EngineT, std::size_t StateSize = EngineT::state_size>
void seed_non_deterministically_2nd(EngineT& engine)
{
    using engine_type = typename EngineT::result_type;
    using device_type = std::random_device::result_type;
    using seedseq_type = std::seed_seq::result_type;
    constexpr auto bytes_needed = StateSize * sizeof(engine_type);
    constexpr auto numbers_needed = (sizeof(device_type) < sizeof(seedseq_type))
        ? (bytes_needed / sizeof(device_type))
        : (bytes_needed / sizeof(seedseq_type));
    std::array<device_type, numbers_needed> numbers{};
    std::random_device device{};
    std::generate(std::begin(numbers), std::end(numbers), std::ref(device));
    std::seed_seq seedseq(std::cbegin(numbers), std::cend(numbers));
    engine.seed(seedseq);
}
```

This code has a number of problems.

- It is absurdly complicated for what could rightfully be expected to be a simple task. (Even the author is not absolutely sure it is correct.) It is unrealistic (and unreasonable) to expect a casual user to come up with such a seeding procedure.
- It is not as general as one might hope it is. The *random number engine* requirements do not specify that an engine exposes its `state_size` as the `std::mersenne_twister_engine` does. (The author believes that making this constant part of the requirements would have been a good idea but it is too late for this now.) This forces the user of the function to look up the actual state size in the documentation and provide the correct value as the second template parameter. (One could check for the existence of `E::state_size` and fall back to `sizeof(E)` as a reasonable estimate for third-party types, faithfully assuming they won't allocate memory to hold state on the free store.)
- It is not as accurate as it should be. Assuming that `std::random_device` produces truly independent uniform random numbers, `std::seed_seq` actually *introduces* non-uniformity [1]. (O'Neill provides experimental evidence for this. Anyway, it is clear that a deterministic algorithm can only ever reduce but never increase the entropy of a given seed.)
- It is not as efficient as it could be. While all that's really needed is copying a sequence of random bytes into the internal state of the engine, the `std::random_device` first copies them into a stack-based `std::array` from which the `std::seed_seq` copies them into a heap allocated (!) internal buffer, scrambles them in a (in this case counter-productive) attempt to remove bias until the engine finally copies them to their final destination. To make matters worse, the `std::random_device` does not know in advance how many bytes will be needed. Therefore, on implementations where it reads from a file such as `/dev/urandom` or uses a syscall like `getrandom(2)` on Linux, it cannot buffer the reads efficiently.

The preceding discussion used `std::random_device` as an example as it is the go-to solution for obtaining system entropy for a non-deterministic seed on C++ today. However, since nothing special about `std::random_device` was made use of, the same discussion applies to any uniform random bit generator.

Unfortunately, writing one's own seed sequence isn't a great solution either. First of all, doing something like this shouldn't be necessary for a common use case. Second and more severely, the seed sequence requirements [\[rand.req.seedseq\]](#) mandate a constructor which takes a pair of iterators, another constructor which takes an initializer list and finally a size and a param member function that are supposed to externalize the object's state such that it can be reconstructed again later. While this makes sense for `std::seed_seq` which is deterministic and always constructed from a sequence of integers, it makes no sense for a seed sequence that obtains its values from a system's entropy source. The “best” way to deal with these functions is to either not implement them (which isn't strictly valid C++20) and hope that the standard library won't enforce the type requirements strictly or implement them to unconditionally throw an exception, which is a sad and annoying option.

2.3 The Proposed Solution

With this proposal adopted, properly seeding a random engine could be as simple as this.

```
template <typename EngineT>
void seed_non_deterministically_3rd(EngineT& engine)
{
    std::random_device device{};
    std::seed_adapter adapter{device};
    engine.seed(adapter);
}
```

There would be no unnecessary copies, no unneeded tempering, no dynamic memory allocation, no introduced bias and little chance to get anything wrong on the user's side.

The `std::seed_adapter` would be a class template (the example above using CTAD) that wraps a reference to any uniform random bit generator and provides a `generate` member function which calls through to the generator's call operator. If [P1068R4](#) were to be adopted and `EngineT` implements the proposed *uniform vector random bit generator* concept, the implementation of `std::seed_adapter<EngineT>::generate` would make exactly one call to `EngineT::operator()`. Even if that proposal should not be adopted, a standard library implementation would be free (and encouraged) to optimize this case for its own implementation of `std::random_device` at least.

3 Impact on the Standard

This proposal could be implemented with very little changes, none of which would be breaking anything. No core language changes are needed.

3.1 Refinements and Additions to the Standard Library

The requirements on the type parameter for a random engine's `seed` member function (and the corresponding constructor) should be relaxed such that it only requires the `generate` member function from *seed sequence*. Bolas [2] has argued that even today, a conforming implementation isn't allowed to *call* anything but `generate` and perhaps `size` due to existing complexity requirements. Unfortunately, it may still enforce their presence via type checks.

To express this relaxed requirement, a new named requirement *seed generator* and a corresponding `std::seed_generator` concept are proposed. Only a `generate` member function would be required. The existing *seed sequence* named requirement would be defined as a refinement of it and a `std::seed_sequence` concept added for completeness.

A new class template `std::seed_adapter` is proposed that implements the `std::seed_generator` concept and acts as an adapter for making any uniform (vector) random bit generator usable as the source for seeding any random number generator. The adapter would be non-owning (note that `std::random_device` is not copyable and copying a (deterministic) random number engine would generally be undesirable because the state transitions would be lost, introducing bad randomness in a way that is not necessarily obvious to users.)

This proposal has no dependencies on any other proposal. However, if [P1068R4](#) were adopted, there would be optimization potential for the implementation of the proposed `std::seed_adapter`. The currently proposed wording was chosen to not preclude this optimization but it cannot mandate it either.

The author is not aware of any other proposal that depends on or conflicts with this proposal. In particular, it is orthogonal to [N3547](#). (However, the sample implementation of `std::randomize` in that paper could benefit from the feature suggested in this proposal.)

3.2 Clarification on Random Number Engine Requirements

Currently, table 94 [\[tab:rand.req.eng\]](#) defines the various overloads of the `seed` member function of a type `E` that meets the requirements of *random number engine* in terms of the corresponding constructor and equality operator. For example, `e.seed(q)` is defined to have the effect that “post: `e == E(q)`” and complexity “same as `E(q)`”. This is unfortunate because for a *seed sequence* `q`, two successive calls to `q.generate` (even for ranges of identical size) do not have to produce the same sequence of numbers, even though `std::seed_seq` happens to behave that way. The requirements for *seed sequence* don't mandate this; the respective row in table 93 [\[tab:rand.req.seedseq\]](#) says about `q.generate(rb, re)` (emphasis added):

Does nothing if `rb == re`. Otherwise, fills the supplied sequence `[rb, re)` with 32-bit quantities that depend on the sequence supplied to the constructor **and possibly also depend on the history of `generate`'s previous invocations**.

Therefore, the author believes that it is not intended by the current standard that the following code should be guaranteed to work.

```
template <typename RandEngT, typename SeedSeqT>
void test(RandEngT& engine, SeedSeqT& seedseq)
{
    engine.seed(seedseq);
    assert(engine == RandEngT{seedseq}); // might fire
}
```

Regardless of whether this proposal will be adopted, the wording in table 94 should be updated to make it clear that calling `seed` puts the engine into the same state as calling the corresponding constructor but not create the impression that the assertion in the above example would necessarily hold.

3.3 Deprecation of Existing Features

It is not proposed that any existing library features be deprecated.

Possible candidates for deprecation could be the overload of `E::seed` that takes a single value of type `E::result_type` and the corresponding constructor of a *random number engine* `E`. Deprecating these overloads might be sensible because those functions actually encourage poorly seeding random number engines. On the other hand, they remain useful in

situations where people want to hard-code a specific seed (such as in `std::minstd_rand{42}`) and don't really care about uniformly choosing an initial state from the entire state-space of the engine or know that the space is sufficiently small.

4 Design Decisions

The (updated) proposal takes care to be non-breaking with existing and compatible with potential future versions of the standard library (which might include [P1068R4](#)). For the sake of backwards compatibility, the specification that the `generate` member function of a seed sequence and therefore a seed generator deal in 32bit integers could not be changed.

5 Alternative Solutions

If [P1068R4](#) were adopted, this proposal could eventually be made obsolete by instead simply adding an additional constructor and seed member function to the random number engine requirements which takes a uniform vector random bit generator as lvalue argument and initializes its internal state by calling its `range operator()` instead of the `param` member function as the seed sequence overload does.

If `p` is an lvalue of a type which implements `std::uniform_vector_random_bit_generator` and `E` is a type which meets the requirements of a random number engine, then the expression `E(p)` shall initialize the engine's state via a single call to `p(f, l)` with `f` and `l` being contiguous iterators with a value type that is an unsigned integer. For an lvalue `e` of type `E`, the expression `e.seed(p)` shall have the same effect as `e = E(p)`.

This would be a breaking change as it would strengthen the requirements on an existing named requirement. Of course, the standard library types could implement this nonetheless and the new requirement be made optional by introducing yet another concept or named requirement.

However, the author is concerned that this solution could be confusing because of the ambiguity with the copy constructor.

Making `std::seed_adapter` own the underlying uniform random bit generator instead of holding a reference to it was considered but not perused for the reasons discussed earlier, namely concerns about generators that are not copyable or non-obvious undesired effects of creating such copies in cases where this is possible.

The author notes that – would this proposal be adopted – the seed *sequence* requirement isn't actually used by the standard library itself. Therefore, the standard might as well just not define it anymore.

6 Implementations

The implementation of this proposal would be very simple. A proof-of-concept is available [online](#).

7 Proposed Wording

All proposed changes to the standard mentioned in this section are relative to N4861.

Add to the synopsis of the `<random>` header [\[rand.req.synopsis\]](#) the following three declarations.

```
template <class S>
concept seed_generator = see below;

template <class S>
concept seed_sequence = see below;

template <class U>
class seed_adapter<U>;
```

Add a new section before the existing § 26.6.2.2 [\[rand.req.seedseq\]](#) to define *seed generator*.

Seed generator requirements [\[rand.req.seedgen\]](#)

A *seed generator* is an object that produces a requested number of unsigned integer values i with $0 \leq i < 2^{32}$. The generated numbers may be obtained from a non-deterministic source of entropy.

A class S satisfies the requirements of a seed generator if the expressions shown in the below table are valid and have the indicated semantics and if S also satisfies all other requirements of this section. In that table and throughout this section:

- T is the type named by S 's associated `result_type`;
- q is a value of S ;
- rb and re are mutable random access iterators with an unsigned integer `value_type` of at least 32 bits.

Expression	Return type	Pre/post-condition	Complexity
<code>S::result_type</code>	T	T is an unsigned integer type of at least 32 bits.	compile-time
<code>q.generate(rb, re)</code>	<code>void</code>	Does nothing if <code>rb == re</code> . Otherwise, fills the supplied sequence <code>[rb, re)</code> with 32-bit quantities in a possibly non-deterministic way.	$O(re - rb)$

In the existing section § 26.6.2.2 [\[rand.req.seedseq\]](#), change the beginning of the second paragraph as follows.

A class S meets the requirements of a seed sequence if [it satisfies the requirements of a seed generator and in addition](#), the expressions [...]

In the current table 93, remove the first (`S::result_type`) and the fifth (`q.generate(rb, re)`) row which will already be covered by the *seed generator* requirements.

Replace section 26.6.2.4 [\[rand.req.eng\]](#) paragraph 4.4 by the following sentence with the appropriate cross-reference to the section defining the requirements of *seed generator*.

`q` is an lvalue satisfying the requirements of a seed **sequence generator**;

Add the following two new concepts for `std::seed_generator` and `std::seed_sequence` at appropriate locations.

```
template <
    class SeedGenerator,
    class RandomAccessIterator = add_pointer_t<typename SeedGenerator::result_type>
>
concept seed_generator = __unsigned_integral_least32<typename SeedGenerator::result_type>
    && random_access_iterator<RandomAccessIterator>
    && __unsigned_integral_least32<typename iterator_traits<RandomAccessIterator>::value_type>
    && requires (SeedGenerator& s, RandomAccessIterator f, RandomAccessIterator l) {
        { s.generate(f, l) } -> same_as<void>;
    };

template <
    class SeedSequence,
    class RandomAccessIterator = add_pointer_t<typename SeedSequence::result_type>,
    class InputIterator = RandomAccessIterator,
    class OutputIterator = RandomAccessIterator
>
concept seed_sequence = seed_generator<SeedSequence, RandomAccessIterator>
    && default_initializable<SeedSequence>
    && constructible_from<SeedSequence, InputIterator, InputIterator>
    && constructible_from<SeedSequence, initializer_list<typename SeedSequence::result_type>>
```



```
&& input_iterator<InputIterator>
&& __unsigned_integral_least32<typename iterator_traits<InputIterator>::value_type>
&& output_iterator<OutputIterator, typename SeedSequence::result_type>
&& requires (const SeedSequence& s, OutputIterator out) {
    { s.size() } -> same_as<size_t>;
    { s.param(out) } -> same_as<void>;
};
```

For better readability, the above code uses the hypothetical concept `__unsigned_integral_least32` which *could* be defined like this.

```
template <typename T>
concept __unsigned_integral_least32 = unsigned_integral<T> && numeric_limits<T>::digits >= 32;
```

Add the following new section.

Class template seed_adapter [rand.req.seedadapt]

```
template <uniform_random_bit_generator U>
class seed_adapter
{
public:
    // types
    using result_type = typename U::result_type;

    // constructors
    explicit constexpr seed_adapter(U& gen) noexcept;

    // generating functions
    template <random_access_iterator It>
    void constexpr generate(const It f, const It l)
    requires __unsigned_integral_least32<typename iterator_traits<It>::value_type>;

private:
    U* m_gen; // exposition only
};
```

```
explicit constexpr seed_adapter(U& gen) noexcept;
```

Effects: Stores the address of gen in m_gen.

```
template <random_access_iterator It>
void constexpr generate(const It begin, const It end)
requires __unsigned_integral_least32<typename iterator_traits<It>::value_type>;
```

Preconditions: The address stored in m_gen (still) refers to a valid object.

Effects: Fills the range [begin, end) with 32 bit values obtained from *m_gen.

Complexity: $O(\text{end} - \text{begin})$

Restructure the first seven rows of the current table 94 as follows. It is believed that this preserves the intended meaning of the current wording but avoids confusion.

Expression	Return type	Pre/post-condition	Complexity
E()		Creates an engine with the same initial state as all other default-constructed engines of type E.	$O(\text{size of state})$

E(x)		Creates an engine that compares equal to x.	O(size of state)
E(s)		Creates an engine with initial state determined by s. <u>[Note: For engines with an internal state larger than sizeof(s), there will be impossible states that cannot be created using this constructor. — end note]</u>	O(size of state)
E(q)		Creates an engine with an initial state that depends on a sequence produced by one call to q.generate. <u>[Note: Depending on the behavior of q.generate, this operation might not be deterministic. — end note]</u>	same as complexity of q.generate called on a range of the size of the engine's state
e.seed()	void	Postconditions: e == E(). <u>Equivalent to e = E().</u>	same as E() <u>not worse than e = E()</u>
e.seed(s)	void	Postconditions: e == E(s). <u>Equivalent to e = E(s).</u>	same as E(s) <u>not worse than e = E(s)</u>
e.seed(q)	void	Postconditions: e == E(q). <u>Equivalent to e = E(q).</u>	same as E(q) <u>not worse than e = E(q)</u>

Acknowledgments

Melissa O'Neill has written a series of remarkable blog posts that discuss the problems with seeding random engines in-depth. Although not the initial motivation for the author of this proposal, that blog post provided valuable theoretical and experimental support and the fact that its author had independently suggested basically the same (revision 0) addition to the standard library was very affirming.

The discussions with the principal author of <random>, Walter Brown, were very enlightening and helped the author a lot figuring out the final details of the proposal (revision 0).

Nicol Bolas, Zhihao Yuan and Seth Cantrell have provided valuable feedback on the std-proposals@isocpp.org mailing list (revision 0).

Baum mit Augen's shared experience of struggling to properly seed a std::mt19937 from a std::random_device and the following discussion with Deduplicator (out of which came an earlier version of the code for seed_non_deterministically_2nd) were part of the motivation for writing this proposal [6].

Jonathan Wakely has provided valuable feedback regarding the use-case of custom seed sequence implementations and pointed the author to the current state of this proposal in the library evolution working group.

Changelog

The following changes were made compared to [revision 0](#) of this proposal:

- Instead of proposing modifications to std::random_device, the formerly “alternative solution” of introducing a generic adapter type is now proposed instead. This change is motivated by feedback regarding the usability with types other than std::random_device and compatibility with [P1068R4](#) in particular.
- The named requirements are accompanied by formal C++20 concepts.
- The wording and references were updated to the C++20 standard.
- The title was changed from “Allow Seeding Random Number Engines with std::random_device” to “Efficient Seeding of Random Number Engines” to account for the broadened scope of this proposal.

The feedback given by LEWG regarding the relaxation of the restriction to 32bit integer types was *not* applied (yet) because the author does not know how this could be done in a backwards-compatible (with C++11 to C++20) way. We would have to strengthen an existing named requirement for that.

References

1. Melissa O'Neill, C++ *Seeding Surprises*. 2015-04-16, <http://www.pcg-random.org/posts/cpp-seeding-surprises.html>
2. Nicol Bolas, via `std-proposals@isocpp.org`, 2016-01-03, <https://groups.google.com/a/isocpp.org/d/msg/std-proposals/sF-P4VE2Z3Q/u24T-g-hEgAJ>
3. “Seed `std::mt19937` from `std::random_device`” in: *Code Review*, 2015-10-30, <http://codereview.stackexchange.com/q/109260>